Unit II	Linear Data Structures, searching and sorting	09 Hours

Overview of Array, Array as an Abstract Data Type, Operations on Array, Storage Representation, Multidimensional Arrays[2D, nD], Sparse matrix representation using 2D

Searching: Sequential Search/Linear Search, Binary Search, Fibonacci Search, and Indexed Sequential Search.

Sorting: Concepts- Stability, Efficiency, and Number of Passes, Internal and External Sorting, Bubble sort, Insertion Sort, Selection Sort , Quick Sort, Merge sort

December 2025 Question on Unit II

Q3) a) Define the terms: **[6]**

i) Sparse Matrix ii) Sorting Stability iii) Internal and External Sort

b) Apply **Binary Search** algorithm on the following numbers stored in an array from **A[0] to A[9]** as:

9, 17, 23, 38, 45, 50, 57, 76, 90, 100

to search the number **23 and 90**. Also note the time complexity of each search. **[8]**

OR

Q4) a) Define **ADT for Arrays** and discuss at least **4 operations** that are performed on arrays. **[6]**

b) Explain **Bubble Sort** algorithm and sort the list using bubble sort: **[8]**

7, 4, 10, 6, 3, 12, 1, 8, 2

Enlist the number of passes required to complete the sort.

Unit II

Linear Data Structures, searching and sorting

Overview of Array

Array is referred as the Sequential Organization that means the data in arrays is stored in some Sequence**.**

If we want to store names of all the students in a class we can make use of an array to store the names in sequential form.

Definition of an array

An array is a linear data structure that stores elements of the same data type in a contiguous block of memory, accessed using indices.

Or

Array is a collection of similar type of elements

An array as a **data structure** is defined as a set of pairs (index, value) such that with each index a value is associated.

Prof.Ishwar Bharambe ADYPSOE

index - indicates the location of an element in an array.

value - indicates the actual value of that data element.

Declaration of an array in „C++“: data_types name_of_array[size]

Example- int a[20];

Here, ‘a’ is the name of the array inside the square bracket size of the array is given . This array is of integer type in array ‘a’.

Key Characteristics

1. Fixed size (static allocation)
2. Elements are indexed from 0 to n-1
3. Random access enabled (O(1) time complexity)
4. Efficient storage and access for homogeneous data

Array as an Abstract Data Type

An ADT is a theoretical model that defines:

The data to be stored

The operations allowed on that data without specifying the internal implementation

An array as an ADT defines:

1. A collection of elements of the same type
2. Fixed size (defined at creation)
3. Indexed access using position

AbstractDataType Array

{ **Instances -** An array a of size ,index & total number of elements in the array n.

Operations-

Create ( ) - This operation create an array.

Insert ( ) - This operation is for inserting elements in the array.

Delete ( ) - This operation is for deleting elements from the array of logical deletion of element is possible.

Display ( ) - This operation is for display the elements of the array

}

Advantages of Array as ADT

1. Random Access
2. Simplicity
3. Memory Locality

Prof.Ishwar Bharambe ADYPSOE

Disadvantages of Array as ADT

1. Fixed Size
2. Costly Insertions/Deletions
3. Wasted Memory

Operations on Array:-

1. Insertion of elements in array
2. Traversing List
3. Deleting an element from array
4. Updating an element in array
5. Searching an element in array

1. Insertion of elements in array

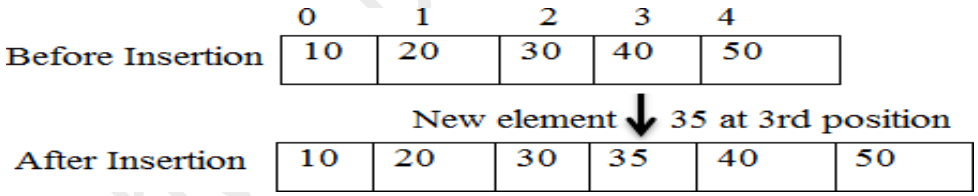
Insert an element in an array is complex activity because we have to shift the elements ahead in order to create a space for new element.

That means after Inserting an element array size gets increase by one.

Time Complexity:

Best Case (end insertion): O(1)

Worst Case (front insertion): O(n)



Insert an element in an array is complex activity because we have to shift the elements ahead in order to create a space for new element. That means after Inserting an element array size gets increase by one.

We can implement array using python program. Python doesn‟t support the concept of array but we can implement the array using lists. List is a sequence of values.

String is also a sequence of values but in string this sequence is of characters. On other-hand, in case of list the values can be of any type.

The values in the list are called elements or items. These elements are separated by comma & enclosed with square brackets.

Example – a=[10,20,30,40] #list of integers

B=[“aaa”,”bbb”,”ccc”] #list of strings

Prof Ishwari Bharambe ADYPSOE

Python Program

```
n=int(input("Enter how many elements you want insert"))
a=[]
i=0
for i in range(n):
    item=int(input("Enter element in array"))
    a.append(item)
possition=int(input("Enter loation where you  want to insert  an elements"))
value=int(input("Enter value to insert"))
a=a[:possition]+[value]+a[possition:]
print("Resultant array is ==>",a)
```

2. Traversing List

Visiting each element of the array from start to end
Used for displaying, summing, or processing all elements
Time Complexity: O(n)

The loop is used in list for traversing purpose. The for loop is used to traverse the list element. Syntax – for variable in List:

Body

Example-

```
a=['a','b','c','d','e']
    for i in a:
        print(i)
```

We can traverse the list using range() function. We can access each element of the list using index of list.

If we want to increment each element of the list by one then we must pass index as argument to for loop. This can be done using range() function as follows.

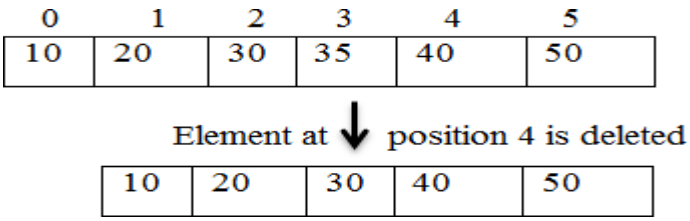
```
a=[10,20,30,40]
for i in range(len(a)):
    a[i] = a[i]
print(a) #     [10,20,30,40]
```

3.Deleting an elements from array

Removing an element from a specific index
Requires shifting elements to the left to fill the gap
Time Complexity: O(n)

Deleting an elements from array is complex activity because have to shift the elements to previous position. That means after Deleting an element size gets decremented by one.

Prof. Ishwar Bharambe ADYPSOE



Program 1: Delete an Element by Value

```
n=int(input("Enter how many elements you want insert"))
a=[]
i=0
for i in range(n):
    item=int(input("Enter element in array"))
    a.append(item)

pos=int(input("Enter location where you  want to delete  an elements"))
a=a[:pos]+a[pos+1:]
print("Resultant array is ==>",a)
```

Program 2: Delete an Element by Value (remove())

```
array = [10, 20, 30, 40, 50]
print("Original List:", array)
array.remove(30)
print("List after deletion:", array)
```

Program 3: Delete an Element by Index (del)

```
# Create a list
numbers = [10, 20, 30, 40, 50]
print("Original List:", numbers)
# Delete the element at index 2
del numbers[2]
print("List after deletion:", numbers)
```

Updating an elements from array

Updating means changing the value of an existing element using its index.

Prof Ishwar Bharambe ADYPSOE

#Python code to Updating an elements from array

```
numbers = [10, 20, 30, 40, 50]
 print("Original list:", numbers)
# Update the element at index 2
numbers[2] = 35
 print("Updated list:", numbers)
```

Time Complexity: O(1)

Searching an elements from array

Finding the position of an element

Types: Linear Search O(n), Binary Search O(log n) (if sorted)

Searching means finding whether an element exists in the list.

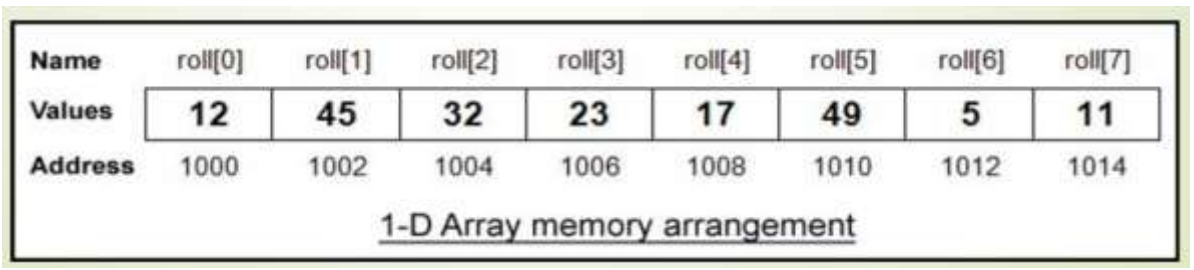
#Python code to Searching an elements from array

```
array = [10, 20, 30, 40, 50]
search = 30
if search in array:
        print(search, "is found in the list.")
else:
        print(search, "is not found.")
```

Storage Representation :-

Storage Representation means How Arrays are Stored in Memory

Arrays are stored in contiguous blocks of memory



1-D Array memory arrangement

Prof.Ishwar Bharambe ADYPSOE

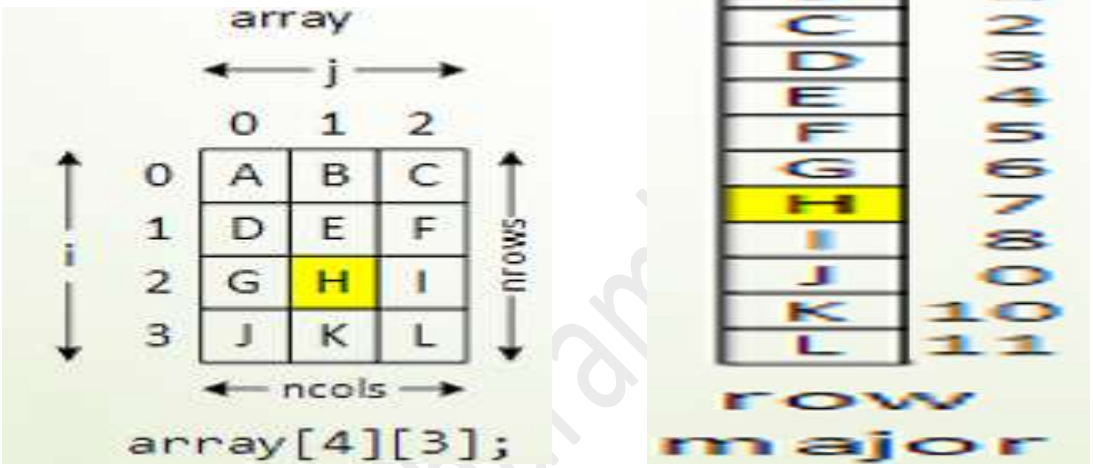
The array can be represented using

i) Row major Representation ii) Column Major Representation

i) Row major Representation

If the elements are stored in row wise manner then it is called as Row major Representation.

Example – if you want to store elements A,B,C,D,E,F,G,H,I,J,K,L then in two dimensional array



The elements will be stored horizontally. To access any elements in two dimensional array we must specify both its row number & column number. That is why we need two variables which is act as row index & column index.

In row major matrix, the element at a[i][j] will be equal to

Address of [i][j] = base address + row_index * total no.of column +column_index)* element-size

Example- Consider integer array int arr[3][4] declared in program. If the base address is 1050, find the address of the element arr[2][3] with row major Representation.

base address + row_index * total no.of column +column_index)* element-size

a[2][3] =1050+(2*4+3)*2 = 1050+(8+3)*2=1050+11*2=1072

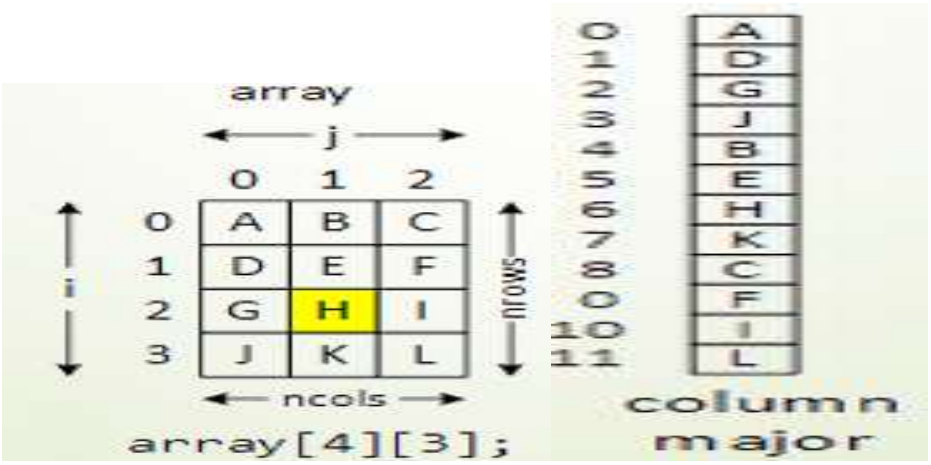
ii) Column Major Representation

If elements are stored in column wise manner then it is called as Column Major Representation.

Example – if you want to store elements A,B,C,D,E,F,G,H,I,J,K,L then in two dimensional array then the elements will be filled up by column wise manner as follows.

Prof. Ishwar Bhajantri CSOE

All elements of the first column are stored first, followed by second, and so on



In Column major matrix, the element at a[i][j] will be equal to

Address of [i][j] = base address + Column _index * total no.of row +row_index)* element-size

Example- Consider integer array int arr[3][4] declared in program. If the base address is 1050, find the address of the element arr[2][3] with Column major Representation.

base address + Column _index * total no.of row +row_index)* element-size

a[2][3] =1050+(3*3+2)*2 = 1050+(9+2)*2=1050+11*2=1072

Address Calculation for One Dimensional Array

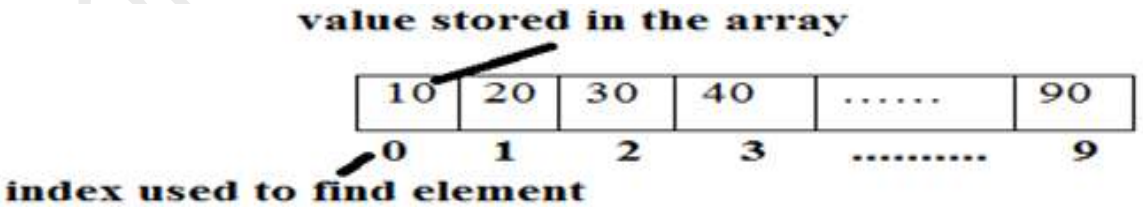
Address of a[i]= base address + i * data_type_size;

Example-

Address of a[3]= 1000 + 3 * 2= 1006.

One dimension:-

The one dimensional array 'a' is declared as int a[10].



Syntax- data_types name_of_array[size] Example- int a[10];

Prof.Ishwar Bharambe ADYPSOE

Two dimensional

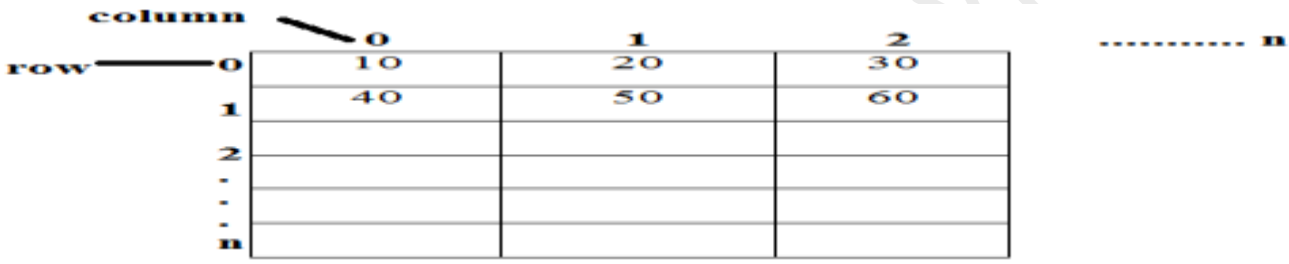
The two dimensional array ‘a’ is declared as int a[3][3]. The two dimensional array should be in row column form.

Syntax- data_types name_of_array[row_size][column_size]

Example- int a[3][4];

Application :-

Image processing, Graph representations, Matrix operations



n- dimensional / Multi- dimensional:-

The n-dimensional array ‘a’ is declared as

int a[3][3]…..[3] .

Syntax-

data_types name_of_array[size1][size2] ….[sizen]

Example- int a[3][3]…..[3];

Application:-

Machine learning (tensors), physics simulations, 3D graphics

Multidimensional Arrays:-

Multidimensional Array is an array having more than one dimension.

Popularly two & three dimensional array are used.

The Multi-dimensional array ‘a’ is declared as

int a[3][3]…..[3] .

Syntax- data_types name_of_array[size1][size2]..[sizen]

Example- int a[3][3]…..[3];

Prof Ishwar Bharambe VPSOE

Sparse Matrix

A matrix can be defined as a two dimensional array having „m" column & „n" rows representing m * n matrix.

Sparse Matrix are those matrices that have the majority of their elements equal to zero. In other words, the Sparse Matrix can be defined as the matrix that has greater number of zero elements than the non-zero element.

Example-

1	0	0
0	0	0
0	1	0

Fig-Sparse Matrix

Sparse matrix representation using array

The representation of Sparse Matrix will be a triplet only. That means it store rows , column and values.

The 0th row will store that rows of the matrix , total column of the matrix & total number of non-zero values.

Suppose a matrix is 4 × 5 & number of non-zero term are say 7.

Row- index of the row where non-zero element is located.

Column - index of the column where non-zero element is located.

Value - value of the non-zero element is located at index.

	0	1	2	3	4
0	0	0	4	0	5
1	0	0	0	3	6
2	0	0	0	2	0
3	0	2	3	0	0



Row	4	0	0	1	1	2	3	3
Column	5	2	4	3	4	3	1	2
value	7	4	5	3	6	2	2	3

Introduction to Searching and Sorting

Searching

Search Techniques is essential for locating the position of the required elements from the heap of data.

Sorting

Sorting is useful for arrange the data in desired order.

Prof. Ishwari Kharambe ADYPSOE

Search Techniques

When we want to find out particular record efficiently from the give list of elements then there are various methods of searching that element. These methods are called as “Searching methods”.

Various algorithm based on these Searching methods are known as searching algorithm.

Characteristics of any search algorithm:-

1) It should be efficient.
2) Less number of computations must be involved.
3) The space occupied by searching algorithm must be less.

The most common use searching algorithms are

1. Sequential Search/Linear Search
2. Binary Search,
3. Fibonacci Search,
4. Indexed Sequential Search.

The element to be searched from the given list is called as the key element.

1) Sequential Search/Linear Search-

Sequential Search/Linear Search is the technique in which given list of elements is scanned from the beginning. The key element is compared with every element of the list. If the match is found the searching is stopped otherwise it will be continued to the end of the list.

Although this is a simple method there is some unnecessary comparison involved in this method.

The time complexity of this algorithm is O(n). The time complexity will increase linearly with value of n.

Algorithm-

1.Start.
2.Read the array arr[], the number of elements n, and the search key as key.
3.Set index i = 0.
4.Repeat while i < n:
5.Compare if arr[i] == key:
Display "Key is found at position i."
6.Otherwise, increment i by 1.
7.If the loop completes without finding the key , Display "key is not found."
8.Stop.

Prof.Ishwar Bhirambe ADYPSOE

Example – Array: [10, 14, 15, 26, 27, 31] Assume Search Key = 26 number of elements = 6

Pass 1 :- i=0

Index (i)	**0**	**1**	**2**	**3**	**4**	**5**
Array	**10**	**14**	**15**	**26**	**27**	**31**

i=0

while(i<6): 0<6 True so check element at i^{th} location value with key

If a[i]== Key i.e. a[0] = 10

i.e. 10 ==26 False so increment i by 1 so i=1

Pass 2 i=1

Index (i)	**0**	**1**	**2**	**3**	**4**	**5**
Array	**10**	**14**	**15**	**26**	**27**	**31**

i=1

while(i<6): 1<6 True so check element at i^{th} location value with key

If a[i]== Key i.e. a[1] = 14

i.e. 14 ==26 False so increment i by 1 so i=2

Pass 3 i=2

Index (i)	**0**	**1**	**2**	**3**	**4**	**5**
Array	**10**	**14**	**15**	**26**	**27**	**31**

i=2

while(i<6): 2<6 True so check element at i^{th} location value with key

If a[i]== Key i.e. a[2] = 15

i.e. 15 ==26 False so increment i by 1 so i=3

Pass 4 i=3

Index (i)	**0**	**1**	**2**	**3**	**4**	**5**
Array	**10**	**14**	**15**	**26**	**27**	**31**

i=3

while(i<6): 3<6 True so check element at i^{th} location value with key

If a[i]== Key i.e. a[3] = 26

i.e. 26 ==26 True so Display message that search key is found at index 3

Prof Ishwar Bharambe ADYPSOE

Advantages of Sequential Search/Linear Search-

1. It is simple to implement
2. It does not required specific ordering before applying the method.

Disadvantages of Sequential Search/Linear Search-

1. It is less efficient.

Real-life application of Linear Search.

1. Searching for a contact in an unsorted contact list.
2. Finding a product in a small inventory.

Time Complexity of Linear search

Best Case Complexity - In Linear search, best case occurs when the element we are finding is at the first position of the array. The best-case time complexity of linear search is **O(1).**

Average Case Complexity - The average case time complexity of linear search is **O(n).**

Worst Case Complexity - In Linear search, the worst case occurs when the element we are looking is present at the end of the array. The worst-case in linear search could be when the target element is not present in the given array, and we have to traverse the entire array. The worst-case time complexity of linear search is **O(n).** The time complexity of linear search is **O(n)** because every element in the array is compared only once.

Space complexity of linear search-

The space complexity of linear search is O(1).

Binary Search-

Binary Search is a searching algorithm in which the list of elements is divided into two sub-list & the key element is compared with the middle element.

If the match is found then , the location of the middle element is returned otherwise we search into either the halves (sub-list) depending upon the result produced through match.

Algorithm Binary Search

1. Start.
2. Read the sorted array arr[], lower index low, upper index high, and the search key key.
3. Repeat while low <= high:

   Calculate the middle index:

   mid = (low + high) / 2

   Compare if arr[mid] == key:

   Return mid (element found).

   else if arr[mid] < key:

   Set low = mid + 1 (search in the right half).

   else:

   Set high = mid - 1 (search in the left half).

4.If low > high, the element is not present in the array.

5.Return -1.

6.Stop.

Example - **All elements should be sorted.**

Array: [-40, 11, 33, 37, 42, 45, 99, 100] Assume search Key=100

Pass 1

Index	0	1	2	3	4	5	6	7
Array	-40	11	33	37	42	45	99	100

low=0, high=7

while(low<=high): 0<=7 → TRUE

Calculate the middle index as mid=low+high /2 =(0+7)/2 = 3

arr[3]=37

if arr[mid] == key

37==100

else if arr[mid] < key

37 < 100 → True so Search only in Right Half of the array

Now set low index as low=mid+1=4

Prof.Ishwar Bharambe ADYPSOE

Pass 2:-

Index	0	1	2	3	4	5	6	7
Array	-40	11	33	37	42	45	99	100

low=4, high=7

while(low<=high): 4<=7 →TRUE

Calculate the middle index as mid=low+high /2 so mid=(4+7)/2 = 5

arr[5]=45

if arr[mid] == key

45==100

else if arr[mid] < key

45 < 100 → True so Search only in Right Half of the array

Now set low index as low==mid+1=5+1=6

Pass 3:

Index	0	1	2	3	4	5	6	7
Array	-40	11	33	37	42	45	99	100

low=6, high=7

while(low<=high): 6<=7 →TRUE

Calculate the middle index as mid=low+high /2 so mid=(6+7)/2 = 6

arr[6]=99

if arr[mid] == key i.e. 99==100

else if arr[mid] < key

99 < 100 → True so Search only in Right Half of the array

Now set low index as low==mid+1=6+1=7

Pass 4:-

Index	0	1	2	3	4	5	6	7
Array	-40	11	33	37	42	45	99	100

low=7, high=7

while(low<=high): 7<=7 → TRUE

Calculate the middle index as mid=low+high /2 so mid=(7+7)/2 = 7

arr[7]=100

if arr[mid] == key

100==100 → TRUE so the 100 is Element found at index 7.

Prof.Ishwar Bharambe ADYPSOE

Advantages of Binary Search-

It is efficient technique

Disadvantages of Binary Search

1) It requires specific ordering before applying the method

2) It is complex to implement

Real-life application of Binary Search.

1. Searching products in a sorted e-commerce database.
2. Finding a word in a dictionary.
3. Searching names in a sorted employee database.

Time Complexity of Binary search-

Best Case Complexity - In Binary search, best case occurs when the element to search is found in first comparison, i.e., when the first middle element itself is the element to be searched. The best-case time complexity of Binary search is **O(1).**

Average Case Complexity - The average case time complexity of Binary search is **O(logn).**

Worst Case Complexity - In Binary search, the worst case occurs, when we have to keep reducing the search space till it has only one element. The worst-case time complexity of Binary search is **O(logn).**

Space complexity of binary search

The space complexity of binary search is O(1).

Comparison between Linear Search and Binary Search

Linear search	**Binary search**
The linear search starts searching from the first element and compares each element with a searched element till the element is not found.	It finds the position of the searched element by finding the middle element of the array.
In a linear search, the elements don't need to be arranged in sorted order.	The pre-condition for the binary search is that the elements must be arranged in a sorted order.
It is based on the sequential approach.	It is based on the divide and conquer approach.
It is preferrable for the small-sized data sets.	It is preferrable for the large-size data sets.
It is less efficient in the case of large-size data sets.	It is more efficient in the case of large-size data sets.
It is less complex.	It is more complex.
It is very slow process.	It is very fast process
It is also called sequential search.	It is also called half-interval search.

Fibonacci Search –

In Fibonacci Search we consider the indices as the number from Fibonacci Series.

As we know the Fibonacci Series

0 1 1 2 3 5 8 13 21 34 ……..

Fibonacci Search algorithm-

1.find f(k) { k^{th} Fibonacci number which is greater than or equal to the n}.

2. if (f(k) = 0)

Stop & print message as the element not found.

3. offset = -1

4. i = min((offset + f(k-2)),(n-1))

5. if key == a[i]

Return i & stop search

6. if if key > a[i]

k = k-1, offset = i & repeat step 4 & 5.

7. if key < a[i]

k = k-2 & repeat step 4 & 5.

Example:-

0	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**
10	**22**	**35**	**40**	**45**	**50**	**80**	**82**	**85**	**90**	**100**

Step 1: Find Fibonacci number ≥ n

k	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**
f(k)	**0**	**1**	**1**	**2**	**3**	**5**	**8**	**13**

find f(k) { k^{th} Fibonacci number which is greater than or equal to the 11 }.

So we find Fibonacci number f(k) =13.

Step 2 :- Since f(k) ≠ 0**, f(13)** ≠ 0 **continue.**

Step 3 Set offset = -1

Pass 1:- Step 4 Find i=min(offset + f(k−2), n−1)

n=11 key = 85 k=7

i = min((offset + f(7-2)),(11-1))

= min((-1 + f(5)),(10))

= min((-1 + 5),(10))

i= min(4,10)=4

Prof Ishwar Bharambe ADYPSOE

if key == a[i] i.e. 85 == a[4] i.e. 85 ==45 false go to next step

if if key > a[i] i.e. 85 > 45 true so

Update: k and Offset

k = k-1= 7-1=6

offset = 4 **& repeat step 4 & 5.**

Pass 2:- k = 6 and offset = 4

i = min((offset + f(k-2)),(n-1))

= min((4 + f(6-2)),(11-1))

= min((4 + f(4)),(10))

= min((4 + 3),(10))

= min(7,10)

i=7

if key == a[i] i.e. 85 == a[7] i.e. 85 ==82 false go to next step

if if key > a[i] i.e. 85 > 82 true so

Update: k and Offset

k = k-1= 6-1=5

offset = i=7 & **repeat step 4 & 5.**

Pass 3:- k = 5 and offset = 7

i = min((offset + f(k-2)),(n-1)) = min((7+ f(5-2)),(11-1))
= min((7 + f(3)),(10)) = min((7+ 2),(10))
=min(9,10)
i=9
if key == a[i] i.e. 85 == a[9] i.e. 90 ==85 false go to next step
if key > a[i] i.e. 85 < 90 true so

Update: k

k = k-2= 5-2=3

offset = 7 & **repeat step 4 & 5.**

Pass 4: k = 3 and offset =7

i = min((offset + f(k-2)),(n-1))
= min((7+ f(3-2)),(11-1)) = min((7 + f(1)),(10))= min((7+ 1),(10))= min(8,10)
i=8
if key == a[i] i.e. 85 == a[8] i.e. 85 ==85
true Return i & stop search
i.e. element is found at index i =8.

Prof. Ishwar Bharambe ADYPSOE

Indexed Sequential Search

In Indexed Sequential Search technique a separate table containing the indices of the actual elements is maintained

The actual search of element is done with the help of this index table

Indexed Sequential Search algorithm-

1. Read the search element from the user
2. Divide the array into groups according to the group size
3. Create index array that contains the staring index of the group.
4. If group is present & first element of that group is less than or equal to the Search key
   Go to the next group
   Else apply linear Search on the previous group.
5. Repeat step 4 for all groups.

Example

Index	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**
value	**12**	**15**	**17**	**21**	**28**	**36**	**57**	**81**	**99**

Assume the **group size = 3** and the **search key = 28**.
Step 1: Read the search element as Key = 28
Step 2: Divide the array into groups

Group	**Elements**
G1	12, 15, 17
G2	21, 28, 36
G3	57, 81, 99

Step 3: Create the index array
The index array contains the **starting index** of each group.

Group	**Starting Index**	**First Element**
G1	**0**	**12**
G2	**3**	**21**
G3	**6**	**57**

Index Array = [0, 3, 6]

Step 4: Compare the first element of each group

Check Group G1

First element = 12

Is 12 ≤ 28? Yes

Go to the next group.

Check Group G2

First element = 21

Is 21 ≤ 28? Yes

Go to the next group.

Prof.Ishwar Bhojane ADYPSOE

Check Group G3

First element = 57

Is 57 ≤ 28? No

According to the algorithm: Apply Linear Search on the previous group (G2)

Step 5: Linear Search in Group 2

Group 3 elements:

Index	Value
3	**21**
4	**28**
5	**36**

Comparisons:

Compare 21 with 28 → Not found.

Compare 28 with 28 → Found ✓

So we found the element 28 at index = 4.

Sorting: Concepts:- Definition :

Sorting is systematic arrangement of data.

Why Sorting?

Sorting organizes data in a specific order (ascending/descending), making searching and processing faster and more efficient



Sort Order

The sorting is a technique by which we expect the list of elements to be arranged as we expect.

Sorting order is nothing but the arrangement of the elements in some specific manner.

Usually the sorting order is of two types –

1) Ascending order:

It is the sorting order in which the elements are arranged from low value to high value. In other words elements are in increasing order.

For example : 10, 50, 40, 20, 30 can be arranged in ascending order after applying some sorting technique as 10, 20, 30, 40, 50

Prof. I... Bharambe ADYPSOE

Descending Order:

It is the sorting order in which the elements are arranged from high value to low value. In other words elements are in decreasing order. It is reverse of the ascending order. For example : 10, 50, 40, 20, 30 can be arranged in descending order after applying some sorting technique as 50, 40, 30, 20, 10

Stability:-

A sorting algorithm is stable if it preserves the relative order of equal elements.

The sorting stability means comparing the records of same value and expecting them in the same order even after sorting them

Example1: In student list, two students with the same marks maintain their input order

For example2 : (Pune, BalGandharva)
(Pune, Shaniwarwada)
(Nasik, Panchavati)
(Mumbai, Gateway-of-India)

Now in the above list, we will sort the list according to the first alphabet of the city. The ascending order for the alphabets P(for Pune), N(for Nasik), M(for Mumbai) will be M, N, P.

The sorting stability can be achieved by arranging the records as follows.

(Mumbai, Gateway-Of-India)
(Nasik, Panchavati)
(Pune, BalGandharva)
(Pune, Shaniwarwada)

Efficiency:-

We usually denote the efficiency of sorting algorithms in terms of time complexity and space complexity.

The time complexities are given in terms of big-on notations.

Number of Passes,

The phases in which the elements are moving to acquire their proper position is called **passes**.

For example : 10, 30, 20, 50, 40

Pass 1 : 10, 20, 30, 50, 40
Pass 2 : 10, 20, 30, 40, 50

In the above method we can see that data is getting sorted in two passes distinctly.

By applying a logic as comparison of each element with its adjacent element gives us the result in two passes.

Prof. Ishwar Bharambe ADYPSOE

Internal Sorting:-Definition

Internal sorting is a sorting method in which all the data to be sorted **fits completely into the computer's main memory (RAM).**

Since the entire dataset is available in memory, the CPU can access it directly, making internal sorting much faster than external sorting.

Example :- Suppose you have an **array: 45, 12, 89, 23, 7**

Since all elements fit into RAM, you can use algorithms like Bubble Sort, Merge Sort, or Quick Sort to sort them.

After sorting: 7, 12, 23, 45, 89

Characteristics of Internal Sorting

1. Entire data is stored in RAM.
2. Fast processing speed.
3. No need for secondary storage during sorting.
4. Suitable for small to medium-sized datasets.
5. CPU directly accesses all data.

Advantages

1. Very fast execution.
2. Simple implementation.
3. Low disk I/O operations.
4. Efficient for datasets that fit in memory.

Disadvantages

1. Cannot handle extremely large datasets.
2. Limited by available RAM.
3. Performance decreases if memory becomes insufficient.

External Sorting: Definition

External sorting is a sorting method used when the data is too large to fit into the computer's main memory (RAM).

The data is stored on secondary storage such as: Hard Disk (HDD), SSD, Magnetic Tape, External Storage

The sorting process repeatedly reads data from disk into memory, sorts it, and writes it back.

Prof. Ishwar Bharambe ADYPSOE

Characteristics of External Sorting

1. Used for huge datasets.
2. Depends on secondary storage.
3. High disk I/O operations.
4. Slower than internal sorting.
5. Memory-efficient for large files.

Advantages

1. An sort files of virtually any size.
2. Suitable for big data.
3. Used in database systems.
4. Handles millions or billions of records.

Disadvantages

1. Slower because disk access is much slower than RAM.
2. More complex implementation.
3. Requires additional storage space.

Difference Between Internal and External Sorting

Feature	**Internal Sorting**	**External Sorting**
Memory Used	Main Memory (RAM)	Secondary Storage (Disk/SSD)
Data Size	Small to Medium	Very Large
Speed	Fast	Slower
Disk Access	Not Required During Sorting	Required Frequently
Cost	Lower	Higher
Complexity	Simpler	More Complex
Storage Requirement	Only RAM	RAM + Disk
Performance	High	Lower due to I/O
Common Algorithm	Quick Sort, Merge Sort, etc.	External Merge Sort

Prof. Ishwar Bharambe ADYPSOE

Sorting Techniques

As mentioned above sorting is an important activity and every time we insert a delete the data we need to sort the remaining data. Therefore it should be carried (a efficiently. Various algorithms are developed for sorting such as

1. Bubble Sort 2. Selection Sort 3. Insertion Sort
4. Merge Sort 5. Quick Sort etc.

Bubble Sort

This is the simplest kind of sorting method in this method. We do this bubble sort procedure in several iterations, which are called passes.

Bubble Sort Algorithm-

Step 1. Read the total number of elements say n

Step 2. Set i=0

Step 3. Repeat step 4 to 5 for i<n-1

Step 4. Set j=0

Step5. Repeat step 6 for j < n-i-1

Step 6. If a[j] > a[j+1] then

Set temp= a[j]

Set a[j]=a[j+1]

Set a[j+1]=temp

Step 7. Print the sorted array of elements

Step 8. Stop

Example [14, 33, 27, 35, 10]

Pass 1 (i = 0) [14, 33, 27, 35, 10]

Step 1: Compare 14 and 33
14 > 33 → False so No Swap
Array: [14, 33, 27, 35, 10]

Step 2: Compare 33 and 27
33 > 27 True→ Swap
Array: [14, 27, 33, 35, 10]

Step 3: Compare 33 and 35
33 >35 → False so No Swap
Array: [14, 27, 33, 35, 10]

Step 4: Compare 35 and 10
35 > 10 True → Swap
Array: [14, 27, 33, 10, 35]

Array after Pass 1: [14, 27, 33, 10, 35]

Prof.Ishwar Bharambe ADYPSOE

Pass 2 (i = 1) : [14, 27, 33, 10, 35]

Step 1: Compare 14 and 27

14 > 27 →False so No Swap

Array: [14, 27, 33, 10, 35]

Step 2: Compare 27 and 33

27 > 33 →False so No Swap

Array:[14, 27, 33, 10, 35]

Step 3: Compare 33 and 10

33 > 10 → True Swap

Array:[14, 27, 10, 33, 35]

Step 4: Compare 33 and 35

33 > 35 →False so No Swap

Array:[14, 27, 10, 33, 35]

Array after Pass 2: [14, 27, 10, 33, 35]

Pass 3 (i = 2) [14, 27, 10, 33, 35]

Step 1: Compare 14 and 27

14 > 27 → False so No Swap

Array:[14, 27, 10, 33, 35]

Step 2: Compare 27 and 10

27 > 10 →True Swap

Array:[14, 10, 27, 33, 35]

Step 3: Compare 27 and 33

27 > 33 → False so No Swap

Array:[14, 10, 27, 33, 35]

Step 4: Compare 33 and 35

33 > 35 → False so No Swap

Array:[14, 10, 27, 33, 35]

Array after Pass 3: [14, 10, 27, 33, 35]

Prof Ishwar Bharambe ADYPSOE

Pass 4 (i = 3) [14, 10, 27, 33, 35]

Step 1: Compare 14 and 10

14 > 10 → True So Swap

Array: [10, 14, 27, 33, 35]

Step 2: Compare 14 and 27

14 > 27 →False So No Swap

Step 3: Compare 27 and 33

27 > 33 → False So No Swap

Step 4: Compare 33 and 35

33 > 35 → False So No Swap

Array after Pass 4:

[10, 14, 27, 33, 35]

Final Sorted [10, 14, 27, 33, 35]

Pass 5 (i = 4) [14, 10, 27, 33, 35]

The j for loop condition becomes false so exit and the elements are sorted

Time Complexity of bubble sort

Best Case Complexity - It occurs when there is no sorting required, i.e. the array is already sorted. The best-case time complexity of bubble sort is **O(n).**

Average Case Complexity - It occurs when the array elements are in jumbled order that is not properly ascending and not properly descending. The average case time complexity of bubble sort is **$O(n^2)$.**

Worst Case Complexity - It occurs when the array elements are required to be sorted in reverse order. That means suppose you have to sort the array elements in ascending order, but its elements are in descending order. The worst-case time complexity of bubble sort is **$O(n^2)$**

Space complexity

The space complexity of bubble sort is O (1). It is because, in bubble sort, an extra variable is required for swapping.

Prof Ishwar Bharambe ADYPSOE

Insertion Sort

In this method the elements are inserted at their appropriate place. Hence is the name insertion sort.

Insertion sort algorithm

1. Consider the first element to be sorted and the rest to be unsorted.
2. Take the first element in the unsorted part(u1) & compare it with sorted part elements(s1).
3. if u1<s1 then insert ul in the correct index, else leave it as it is.
4. Take next element in the unsorted part
5. Repeat 3 and 4 until all the elements are sorted.

Example:- Array =[25,17, 31, 13, 2]

Consider the first element to be sorted and the rest to be unsorted.

Sorted Part: 25

Unsorted Part: 17, 31, 13, 2

Take the first element in the unsorted part(u1) & compare it with sorted part elements(s1).

if u1<s1 then insert ul in the correct index, else leave it as it is.

Pass 1

- Take the first unsorted element 17 if u1<s1 then insert ul in the correct index
- Compare Since 17 < 25, shift 25 one position to the right.
- Insert 17 before 25.

Sorted Part: 17, 25

Unsorted Part: 31, 13, 2



Pass 2:

Take the next unsorted element 31. if u1<s1 then insert ul in the correct index

Compare 31 with 25.

Since 31 > 25, it is already in the correct position. No shifting is required.

Sorted Part: 17, 25, 31

Unsorted Part: 13, 2



Prof.Ishwar Bharambe ADYPSOE

Pass 3:

Take the next unsorted element 13. if u1<s1 then insert ul in the correct index

Compare 13 < 31, so shift 31 to the right.

Compare 13 < 25, so shift 25 to the right.

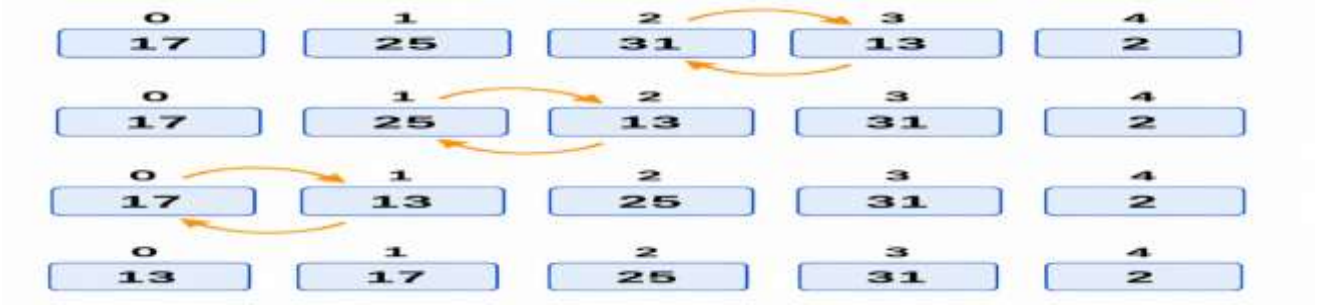
Compare 13 < 17, so shift 17 to the right.

Insert 13 at the first position.

Sorted Part: 13, 17, 25, 31

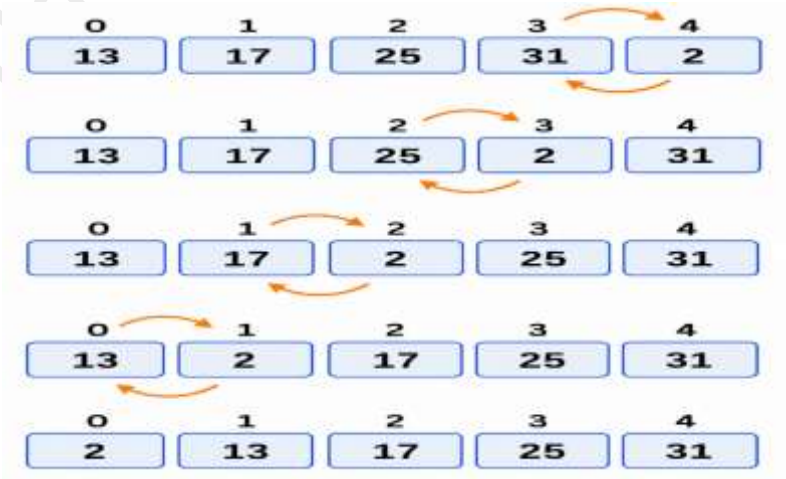
Unsorted Part: 2

Array after Pass 3: 13, 17, 25, 31, 2



Pass 4

- Take the last unsorted element 2.
- Compare 2 with 31.
  - 2 < 31, so shift 31 to the right.
- Compare 2 with 25.
  - 2 < 25, so shift 25 to the right.
- Compare 2 with 17.
  - 2 < 17, so shift 17 to the right.
- Compare 2 with 13.
  - 2 < 13, so shift 13 to the right.
- Insert 2 at the first position.



Sorted Part: 2, 13, 17, 25, 31

Unsorted Part: None

Array after Pass 4: 2, 13, 17, 25, 31

Time Complexity of insertion sort

Best Case Complexity - It occurs when there is no sorting required, i.e. the array is already sorted. The best-case time complexity of insertion sort is **O(n)**.

Prof Ishwar Bharamgouda BDSOE

Average Case Complexity - It occurs when the array elements are in jumbled order that is not properly ascending and not properly descending. The average case time complexity of insertion sort is **$O(n^2)$**.

Worst Case Complexity - It occurs when the array elements are required to be sorted in reverse order. That means suppose you have to sort the array elements in ascending order, but its elements are in descending order. The worst-case time complexity of insertion sort is **$O(n^2)$**.

2. Space Complexity of insertion sort

The space complexity of insertion sort is O (1).

Selection Sort

Scan the array to find its smallest element & swap it with the first element then starting with the second element scan the entire list to find the smallest element & swap it the second element. Then staring from the third element the entire list is scanned in order to find the next smallest element. Continuing in this fashion we can sort the entire list.

Selection Sort Algorithm (Ascending order)

1. **Start**
2. Read the number of elements n.
3. Read the array arr containing n elements.
4. **For** i = 0 **to** n - 1

   1.do Set Min = i.

   2.For j = i + 1 **to** n - 1 **do**

   If arr[j] < arr[Min] **then**

   Set Min = j.

   3.Swap arr[i] and arr[Min] using a temporary variable:

   temp = arr[i]

   arr[i] = arr[Min]

   arr[Min] = temp

5.Display the sorted array.

6.Stop

Prof. Ishwar Bharambe ADYPSC

Example:- 14,33,27,10,35,19

Pass 1 (i = 0)

- Assume Min = 0 → 14
- Compare with remaining elements:
    - 33 < 14 → False so No change
    - 27 < 14 → False so No change
    - 10 < 14 → True so set Min = j=3
    - 35 < 10 → False so No change
    - 19 < 10 → False so No change

We found minimum element at location j=3 so swap it with i=0

Swap arr[0] and arr[3]

Before Swap: 14,33,27,10,35,19

After Swap: 10,33,27,14,35,19

Pass 2 (i = 1) 10,33,27,14,35,19

Assume Min = i=1 → 33

Compare:

27 < 33 → True so set Min = j=2

14 < 27 → True so set Min =j= 3

35 < 14 → False so No change

19 < 14 → False so No change

We found minimum element at location j=3 so swap it with i=1

Swap arr[1] and arr[3]

Before Swap: 10,33,27,14,35,19

After Swap: 10,14,27,33,35,19

Pass 3 (i = 2) 10,14,27,33,35,19

Assume Min = i=2 → 27

Compare:

33 < 27 →False so No change

35 < 27 → False so No change

19 < 27 →True so set Min =j= 5

Swap arr[2] and arr[5]

We found minimum element at location j= 5 so swap it with i=2

Before Swap: 10,14,27,33,35,19

After Swap: 10,14,19,33,35,27

Prof.Ishwar Bharambe

Pass 4 (i = 3)

Assume Min =i= 3 → 33

Compare:

35 < 33 →False so No change

27 < 33 →True so set Min =j= 5

We found minimum element at location j= 5 so swap it with i=3

Swap arr[3] and arr[5]

Before Swap: 10,14,19,33,35,27

After Swap: 10,14,19,27,35,33

Pass 5 (i = 4)

Assume Min =i= 4 → 35

Compare:

33 < 35 →True so set Min =j= 5

We found minimum element at location j= 4 so swap it with i=5

Swap arr[4] and arr[5]

Before Swap: 10,14,19,27,35,33

After Swap: 10,14,19,27,33,35

Pass 6 (i = 5)

Only one element remains.

No swap required.

Final Array:

[10, 14, 19, 27, 33, 35, 42, 44]

Time Complexity of selection sort-

Best Case Complexity - It occurs when there is no sorting required, i.e. th

e array is already sorted. The best-case time complexity of selection sort is **$O(n^2)$**.

Average Case Complexity - It occurs when the array elements are in jumbled order that is not properly ascending and not properly descending. The average case time complexity of selection sort is **$O(n^2)$**.

Worst Case Complexity - It occurs when the array elements are required to be sorted in reverse order. That means suppose you have to sort the array elements in ascending order, but its elements are in descending order. The worst-case time complexity of selection sort is **$O(n^2)$**.

Space Complexity

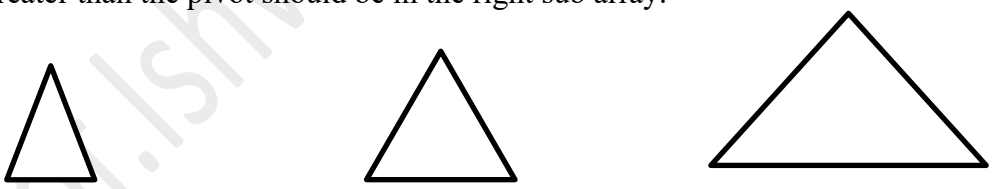
The space complexity of selection sort is O(1). It is because, in selection sort, an extra variable is required for swapping.

Quick Sort

Quick sort is sorting algorithm that uses the divide & conquers strategy. In this method division is dynamically carried out. The three steps of quick sort are as follows

1. Divide-

Split the array into two sub array that each element in the left sub array is less than or equal the middle element in the right sub array is greater than the middle element. The splitting of the array into two sub arrays is based on the pivot element. Al elements that are less than pivot should be in let sub array & all the elements that are greater than the pivot should be in the right sub array.



Elements that are less than pivot Pivot Element Elements that are greater than pivot

2. Conquer –

Reclusively sort the two sub arrays.

3. Combine-

Combine all the sorted elements in a group to form a lsit of sorted elements

It is also called as Partition Exchange Sort developed by tony Hoare in 1959 & published in 1961

Prof. Ishwar Bharambe A.I.T.

Partitioning Algorithm:-

Step 1: Consider the first element of the list as the pivot (i.e., the element at the first position).

Step 2: Take two variables, i and j, to point to elements in the list, excluding the pivot.

Step 3: Set i = low_index + 1.

Step 4: Set j = high_index.

Step 5: While i ≤ j and arr[i] < pivot, increment i.

Step 6: While i ≤ j and arr[j] > pivot, decrement j.

Step 7: If i < j, swap arr[i] and arr[j]. and then increment i and decrement j.

Step 8: Repeat Steps 5–7 until i > j.

Step 9: When i > j, swap the pivot with arr[j]

Step 10: The pivot is now in its correct sorted position. Apply the same process recursively to the left and right subarrays.

Quick Sort Algorithm Steps

Step 1: Check whether low < high. If yes, continue with the following steps.

Step 2: Call the partition method: partition(Array, low, high)

Store the returned partition index in a variable, for example: p = partition(Array, low, high)

Step 3: Make a recursive call to sort the left sub-array: quickSort(Array, low, p - 1)

Step 4: Make a recursive call to sort the right sub-array: quickSort(Array, p + 1, high)

Step 5: Continue this process recursively until low ≥ high.

Quick Sort Algorithm Example:-

Array: [8, 3, 7, 4, 9, 2, 6]
Step 1: Select pivot
We will use the first element as the pivot.
The first element is: **Pivot = 8**
So: [8, 3, 7, 4, 9, 2, 6]
Step 2–4: Initialize i and j
i = low + 1 = 0+1=1
j = high = 6

Index	0	1	2	3	4	5	6
Array	8	3	7	4	9	2	6

Step 5: Move i while arr[i] < pivot.

arr[1] = 3 < 8 → True so increment i

arr[2] = 7 < 8 → True so increment i

arr[3] = 4 < 8 → True so increment i

arr[4] = 9 < 8 → False so Stop. when i = 4

Step 6: Move j

Move j while arr[j] > pivot.

arr[6] = 6 > 8 ❌ → **False so Stop. when j = 6**

Now i <= j, so swap arr[i] and arr[j]. then increment i and decrement j.

Step 7: Swap

Swap: arr[4] = 9 and arr[6] = 6

Array becomes: [8, 3, 7, 4, 6, 2, 9]

Now: **i = 5, j = 5**

Continue the loop Move i

arr[5] = 2 < 8 So increment i. **i = 6**

arr[6]=9 < 8 ❌ → False so Stop. when . i = 6

Move j

arr[5] = 2 > 8 ❌ → **False so Stop. when j = 5**

Now: i > j → 6 > 5

Therefore, according to **Step 8**, swap the pivot with arr[j]. so array is [2, 3, 7, 4, 6, 8, 9]

Second Partition: [2, 3, 7, 4, 6] **Pivot** First element: **Pivot = 2** i = 1, j = 4

Array: [2, 3, 7, 4, 6, 8, 9]

Move i

3 < 2 ❌ → **False so Stop. when . i = i**

Move j

6 > 2 → True so decrease the j = 3

4 > 2 → True so decrease the j = 2

7 > 2 → True so decrease the j = 1

3 > 2 → True so decrease the j = 0

Now: i = 1, j = 0

Since: i > j so, Swap pivot 2 with arr[0].

Nothing changes: [2, 3, 7, 4, 6, 8, 9] Pivot 2 is already in the correct position.

Prof.Ishwar Bharambe ADYPSOE

Third Partition: [3, 7, 4, 6]

Now consider the part after 2: [3, 7, 4, 6] **Pivot = 3** i = 2, j = 4

Move i

7 < 3 ❌ false so stop i = 2.

Move j

6 > 3 → True so decrease the j = 3

4 > 3 → True so decrease the j = 2

7 > 3 → True so decrease the j = 1

Now: i = 2, j = 1

Since i > j, swap pivot 3 with arr[1].

Again, no change: [2, 3, 7, 4, 6, 8, 9]

Fourth Partition: [7, 4, 6] Pivot = 7 i = 3, j = 4

Move i

4 < 7 → **True so increment** i = 4

6 < 7 → **True so increment** i = 5

Move j i > j, so stop.

Swap pivot 7 with arr[j], where j = 4.

Swap 7 and 6. So Array becomes: [2, 3, 6, 4, 7, 8, 9] Now 7 is in its correct position.

Fifth Partition: [6, 4] Pivot = 6 i = 3, j = 3

Move I 4 < 6 → **True so increment** i = 4

Move j 4 > 6 is false. Stop Now: i > j

Swap pivot 6 with arr[3]. Array becomes: [2, 3, 4, 6, 7, 8, 9]

Time Complexity of Quicksort

Best Case Complexity - In Quicksort, the best-case occurs when the pivot element is the middle element or near to the middle element. The best-case time complexity of quicksort is **O(n*logn)**.

Average Case Complexity - It occurs when the array elements are in jumbled order that is not properly ascending and not properly descending. The average case time complexity of quicksort is **O(n*logn)**.

Worst Case Complexity - In quick sort, worst case occurs when the pivot element is either greatest or smallest element. Suppose, if the pivot element is always the last element of the array, the worst case would occur when the given array is sorted already in ascending or descending order. The worst-case time complexity of quicksort is **$O(n^2)$**.

Space complexity of quicksort-

The space complexity of quicksort is O(n*logn).

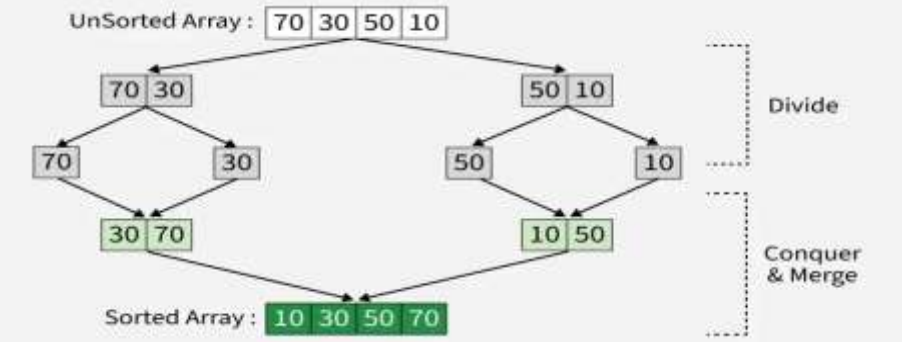
Jawar Bharambe ADYPSOE

Merge Sort:-

- Merge sort is a popular sorting algorithm known for its efficiency and stability.
- It follows the Divide and Conquer approach.
- It works by recursively dividing the input array into two halves, recursively sorting the two halves and finally merging them back together to obtain the sorted array.

Merge Sort Algorithm:-

- Divide: Divide the list or array recursively into two halves until it can no more be divided.
- Conquer: Each sub-array is sorted individually using the merge sort algorithm.
- Merge: The sorted sub-arrays are merged back together in sorted order. The process continues until all elements from both sub-arrays have been merged.



Comparison of All Sorting Methods and their complexities

Sorting technique	Best case	Average case	Worst case	Space Complexity
Bubble sort	O (n)	O (n^2)	O (n^2)	O(1)
Insertion sort	O (n)	O (n^2)	O (n^2)	O (1)
Selection sort	O(n^2)	O (n^2)	O(n^2)	O (1)
Quick sort	O (n log n)	O (n log n)	O (n^2)	O (n log n)

Prof.Ishwar Nagra COVPSOE